

```
from datasets import load_dataset
from tqdm import tqdm
import os, gc

OUTPUT_DIR = "data"
os.makedirs(OUTPUT_DIR, exist_ok=True)
TARGET_LINES = 1_000_000

def download_gujarati():
    dataset = load_dataset(
        "ai4bharat/IndicCorpV2", "indiccorp_v2",
        split="guj_Gujr", streaming=True, trust_remote_code=True,
    )
    sentences = []
    with tqdm(total=TARGET_LINES, desc="Gujarati") as pbar:
        for sample in dataset:
            text = sample.get("text", "").strip()
            if text:
                sentences.append(text)
                pbar.update(1)
            if len(sentences) >= TARGET_LINES:
                break
    with open(os.path.join(OUTPUT_DIR, "gujarati.txt"), "w", encoding="utf-8") as f:
        f.write("\n".join(sentences))
    print(f"Saved {len(sentences)} Gujarati lines.")

def download_english(target_lines=1_000_000, flush_every=50_000):
    dataset = load_dataset("allenai/c4", "en", split="train", streaming=True)
    path = os.path.join(OUTPUT_DIR, "english.txt")
    count = 0
    buffer = []
    with open(path, "w", encoding="utf-8") as f, tqdm(total=target_lines, desc="English") as pbar:
        for sample in dataset:
            text = sample.get("text", "").strip()
            if text:
                buffer.append(text)
                count += 1
                pbar.update(1)
            if len(buffer) >= flush_every:
                f.write("\n".join(buffer) + "\n")
                buffer.clear()
                gc.collect()
            if count >= target_lines:
                break
    if buffer:
        f.write("\n".join(buffer) + "\n")
    print(f"Saved {count} English lines.")

download_english()
```

```
'trust_remote_code' is not supported anymore.
Please check that the Hugging Face dataset 'ai4bharat/IndicCorpV2' isn't base
If the dataset is based on a loading script, please ask the dataset author to
ERROR:datasets.load:`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'ai4bharat/IndicCorpV2' isn't base
If the dataset is based on a loading script, please ask the dataset author to
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
Gujarati: 100%|██████████| 1000000/1000000 [00:54<00:00, 18427.45it/s]
Saved 1000000 Gujarati lines.

Resolving data files: 100%                               1024/1024 [00:00<00:00, 10664.58it/s]

Resolving data files: 100%                               1024/1024 [00:00<00:00, 12650.30it/s]

English (C4): 100%|██████████| 1000000/1000000 [06:33<00:00, 2538.37it/s]Save
```

```
import re

URL_RE = re.compile(r'(https?://\S+|www\.\S+)')
EMAIL_RE = re.compile(r'[\w\.-]+@[ \w\.-]+\.\w+')
DATE_RE = re.compile(r'\b\d{1,2}[/-]\d{1,2}[/-]\d{2,4}\b')
NUM_RE = re.compile(r'\b\d+(?:[.,]\d+)*\b')

SENT_SPLIT_RE = re.compile(r'(?<=[!|.|?])\s+')
PUNCT_RE = re.compile(r'([.,!?:;"\'()\[\]\{\}--\-\...\|\|\|])')

def sentence_tokenize(text):
    text = text.strip()
    if not text:
        return []
    sentences = SENT_SPLIT_RE.split(text)
    return [s.strip() for s in sentences if s.strip()]

def word_tokenize(sentence):
    tokens = []
    protected = []
    def protect(pattern, s):
        def repl(m):
            protected.append(m.group(0))
            return f'__PROT{len(protected)-1}__'
        return pattern.sub(repl, s)

    s = protect(URL_RE, sentence)
    s = protect(EMAIL_RE, s)
    s = protect(DATE_RE, s)

    s = PUNCT_RE.sub(r' \1 ', s)

    s = re.sub(r'(\d)\s*\.\s*(\d)', r'\1.\2', s)

    for tok in s.split():
        if tok.startswith('__PROT') and tok.endswith('__'):
            idx = int(tok[6:-2])
```

```

        tokens.append(protected[idx])
    else:
        tokens.append(tok)
return tokens

```

```

def process_corpus(input_path, output_prefix):
    with open(input_path, encoding="utf-8") as f:
        text = f.read()

    all_sentences = []
    for para in text.split("\n"):
        all_sentences.extend(sentence_tokenize(para))

    with open(f"{output_prefix}_sentences.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(all_sentences))

    with open(f"{output_prefix}_tokens.txt", "w", encoding="utf-8") as f:
        for sent in all_sentences:
            toks = word_tokenize(sent)
            f.write(" ".join(toks) + "\n")

    return all_sentences

```

```

def compute_stats(sentences):
    all_tokens = []
    total_chars = 0
    for sent in sentences:
        toks = word_tokenize(sent)
        all_tokens.extend(toks)
        total_chars += sum(len(t) for t in toks)

    num_sentences = len(sentences)
    num_words = len(all_tokens)
    types = set(all_tokens)

    stats = {
        "total_sentences": num_sentences,
        "total_words": num_words,
        "total_characters": total_chars,
        "avg_sentence_length": num_words / num_sentences if num_sentences else 0,
        "avg_word_length": total_chars / num_words if num_words else 0,
        "type_count": len(types),
        "token_count": num_words,
        "type_token_ratio": len(types) / num_words if num_words else 0,
    }
    return stats

for name in ["gujarati", "english"]:
    sents = process_corpus(f"data/{name}.txt", f"data/{name}")
    stats = compute_stats(sents)
    print(f"\n--- {name.upper()} STATS ---")
    for k, v in stats.items():
        print(f"{k}: {v:.4f}" if isinstance(v, float) else f"{k}: {v}")

```

```
# NLTK
import nltk
nltk.download('punkt')
from nltk.tokenize import sent_tokenize, word_tokenize as nltk_word_tokenize

def nltk_stats(text):
    sents = sent_tokenize(text)
    words = [w for s in sents for w in nltk_word_tokenize(s)]
    types = set(words)
    return {
        "sentences": len(sents),
        "words": len(words),
        "avg_sent_len": len(words)/len(sents),
        "ttr": len(types)/len(words),
    }

import spacy
nlp_en = spacy.load("en_core_web_sm")

def spacy_stats(text, nlp):
    doc = nlp(text)
    sents = list(doc.sents)
    words = [t.text for t in doc if not t.is_space]
    types = set(words)
    return {
        "sentences": len(sents),
        "words": len(words),
        "avg_sent_len": len(words)/len(sents),
        "ttr": len(types)/len(words),
    }
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

✓ Lab 4

```
import random
from collections import defaultdict, Counter
import math

def load_sentences(path):
    sentences = []
    with open(path, encoding="utf-8") as f:
        for line in f:
            toks = line.strip().split()
            if toks:
                sentences.append(toks)
    return sentences
```

```
sentences = load_sentences("data/gujarati_tokens.txt")
sentences = sentences[:1000000]
print(len(sentences))
```

```
1000000
```

```
random.seed(42)
random.shuffle(sentences)

test_data = sentences[:1000]
dev_data = sentences[1000:2000]
train_data = sentences[2000:]

print(len(train_data), len(dev_data), len(test_data))
```

```
998000 1000 1000
```

```
def add_boundaries(sents, n):
    padded = []
    for s in sents:
        toks = ["<s>"] * (n - 1) + s + ["</s>"]
        padded.append(toks)
    return padded
```

```
class NGramModel:
    def __init__(self, n):
        self.n = n
        self.ngram_counts = defaultdict(Counter)
        self.context_counts = Counter()
        self.vocab = set()

    def train(self, sents):
        padded = add_boundaries(sents, self.n)
        for toks in padded:
            self.vocab.update(toks)
            for i in range(self.n - 1, len(toks)):
                context = tuple(toks[i - self.n + 1:i])
                word = toks[i]
                self.ngram_counts[context][word] += 1
                self.context_counts[context] += 1

    def prob(self, context, word):
        V = len(self.vocab)
        c_context = self.context_counts.get(context, 0)
        c_ngram = self.ngram_counts.get(context, {}).get(word, 0)
        return (c_ngram + 1) / (c_context + V)

    def sentence_logprob(self, sent):
        toks = ["<s>"] * (self.n - 1) + sent + ["</s>"]
        logprob = 0.0
        for i in range(self.n - 1, len(toks)):
            context = tuple(toks[i - self.n + 1:i])
            word = toks[i]
```

```

        p = self.prob(context, word)
        logprob += math.log(p)
    return logprob

```

```

def perplexity(self, sents):
    total_logprob = 0.0
    total_words = 0
    for s in sents:
        total_logprob += self.sentence_logprob(s)
        total_words += len(s) + 1
    return math.exp(-total_logprob / total_words)

```

```

unigram = NGramModel(1)
bigram = NGramModel(2)
trigram = NGramModel(3)
quadrigram = NGramModel(4)

```

```
unigram.train(train_data)
```

```
bigram.train(train_data)
```

```

trigram.train(train_data)
quadrigram.train(train_data)

```

```

models = {"unigram": unigram, "bigram": bigram, "trigram": trigram, "quadrigram": quadrigram}

for name, model in models.items():
    dev_pp = model.perplexity(dev_data)
    test_pp = model.perplexity(test_data)
    print(name, "dev:", dev_pp, "test:", test_pp)

```

```

unigram dev: 3707.837253012838 test: 3939.50738814427
bigram dev: 14405.013790203278 test: 14551.475639561195
trigram dev: 96274.37872122774 test: 98539.3061053656
quadrigram dev: 223160.03649373873 test: 230037.31842176704

```

```

context = ("ʌl",)
word = "ð"
print(bigram.prob(context, word))

```

```
0.00041727682535791266
```

```
Start coding or generate with AI.
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.